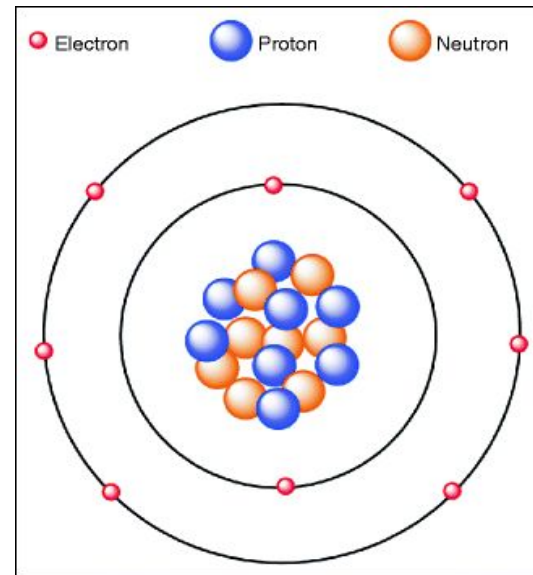


Full Configuration Interaction (FCI)

Presented By - Soham Phulare
Shah and Anchor Kutchhi Engineering College

The Electronic Structure Problem

- The central challenge in computational quantum chemistry is accurately predicting the behavior of interacting electrons to determine molecular properties.
- We apply the Born-Oppenheimer approximation to make this many-body problem tractable. Because atomic nuclei are significantly heavier than electrons, their motion is relatively static.
- This decoupling allows us to treat the nuclei as fixed in space and focus entirely on the movement of electrons within a static electric field.
- The primary goal is to determine the lowest energy solution to this electronic Hamiltonian, known as the ground state, which dictates a molecule's stable configuration and chemical reactivity.



Mathematical Foundations of FCI

- The objective is to find the exact ground-state energy by solving the time-independent Schrödinger equation: $H|\Psi\rangle = E|\Psi\rangle$.
- We restrict the problem to a finite and manageable space by defining a basis set of M available spin-orbitals ($2 * \text{Spatial Orbitals}$) for N electrons.
- A single arrangement of N electrons in M orbitals forms a basis state called a Slater determinant, denoted as $|\Phi_I\rangle$.
- The total number of possible basis states dictates the dimensionality of the problem. This is calculated using the binomial coefficient:

$$\binom{M}{N} = \frac{M!}{N!(M-N)!}$$

Formulating the Eigenvalue Problem

- In the Full Configuration Interaction (FCI) method, the exact wavefunction ($|\Psi\rangle$) is expressed as a complete linear combination of all possible electron configurations:

$$|\Psi\rangle = \sum_I C_I |\Phi_I\rangle$$

- Substituting this expansion into the Schrödinger equation and applying orthonormality ($\langle\Phi_I|\Phi_J\rangle = \delta_{IJ}$) yields:

$$\sum_J \langle\Phi_I|\hat{H}|\Phi_J\rangle C_J = EC_I$$

- This simplifies directly into a standard matrix eigenvalue equation:

$$HC = EC$$

- Solving FCI is strictly equivalent to diagonalizing the Hamiltonian matrix H to find the ground state energy E and the state vector C .

Translating Physics to Code in Qiskit

- Physical properties must be translated into data structures using Python and Qiskit to solve the system computationally.
- `PySCFDriver` defines the physical parameters, including atoms, coordinates, and basis sets, and calculates fundamental electron interactions.
- These interactions form the `fermionic_op`, which represents the exact H matrix.
- The `JordanWignerMapper` translates the complex fermionic behaviors into a tensor product of Pauli matrices that computational frameworks can process natively.

```
from qiskit_nature.second_q.drivers import
PySCFDriver
from qiskit_nature.second_q.mappers import
JordanWignerMapper

driver = PySCFDriver(atom="H 0 0 0; H 0 0 0.735",
basis="6-31g*")
problem = driver.run()

fermionic_op = problem.hamiltonian.second_q_op()

mapper = JordanWignerMapper()
qubit_op = mapper.map(fermionic_op)
```

Exact Classical Diagonalization

- The constructed `qubit_op` matrix is passed to a classical solver, `NumPyMinimumEigsolver`, to perform the $HC = EC$ diagonalization.
- The solver loads the dense matrix into memory to compute the lowest eigenvalue and its corresponding eigenvector.
- The outputs mirror the mathematical formulation. The eigenvalue represents the exact energy (E), and the eigenstate provides the coefficients (C).

```
from qiskit_algorithms import NumPyMinimumEigsolver

exact_solver = NumPyMinimumEigsolver()
exact_result = exact_solver.compute_minimum_eigenvalue(qubit_op)

fci_energy = exact_result.eigenvalue.real
fci_statevector = exact_result.eigenstate
```

Constructing the Hamiltonian Matrix H

- Once `compute_minimum_eigenvalue` is called, the Schrödinger equation simplifies directly to the standard eigenvalue problem:

$$HC = EC$$

- To handle the immense $\binom{M}{N}$ dimensionality of the basis states, the abstract quantum operator `H` is computationally instantiated as a sparse matrix. This prevents memory overload during the calculation, only falling back to a dense representation if the specific operator type strictly requires it.

```
try:
    op_matrix = operator.to_matrix(sparse=True)
except TypeError:
    logger.debug(
        "WARNING: operator of type `%s` does not support sparse matrices. "
        "Trying dense computation", type(operator)
    )
    op_matrix = operator.to_matrix()
```

Solving the Eigenvalue Equation

- Solving the Full Configuration Interaction (FCI) is strictly equivalent to diagonalizing this newly constructed Hamiltonian. Because valid physical observables are Hermitian $H = H^\dagger$, the solver explicitly verifies this symmetry before proceeding.
- This verification allows the algorithm to route the computation through highly optimized mathematical routines (eigh for dense, eigsh targeting the Smallest Algebraic eigenvalues for sparse), mathematically guaranteeing the resulting energies are purely real numbers.

```
def _solve_sparse(op_matrix: scipy.sparse.csr_matrix, k: int)
    -> tuple[np.ndarray, np.ndarray]:
    if (op_matrix != op_matrix.getH()).nnz == 0:
        return scipy.sparse.linalg.eigsh(op_matrix, k=k,
            which="SA")
    else:
        return scipy.sparse.linalg.eigs(op_matrix, k=k,
            which="SR")
```

```
def _solve_dense(op_matrix: np.ndarray) -> tuple[np.ndarray,
    np.ndarray]:
    if op_matrix.all() == op_matrix.conj().T.all():
        return cast(Tuple[np.ndarray, np.ndarray],
            np.linalg.eigh(op_matrix))
```


Isolating the Exact Ground State

- The exact ground-state energy corresponds to the absolute minimum eigenvalue E , with its corresponding eigenvector C representing the exact wavefunction $|\Psi\rangle$.
- Once the core solver computes and sorts the energy spectrum, the minimum eigensolver wrapper strictly extracts the lowest mathematical state.
- This systematically discards all higher-energy excited states, cleanly isolating the exact ground-state solution to return back to the main script.

```
# Sorting the resulting E and C arrays from the core solver
indices = np.argsort(eigval)[: self._k]
eigval = eigval[indices]
eigvec = eigvec[:, indices]

# Extracting the exact ground state (index 0)
if eigensolver_result.eigenvalues is not None and
len(eigensolver_result.eigenvalues) > 0:
    result.eigenvalue = eigensolver_result.eigenvalues[0]
    result.eigenstate = eigensolver_result.eigenstates[0]
```

The Computational Bottleneck

- While mathematically exact, classical diagonalization is severely limited by hardware constraints.
- The Hamiltonian matrix dimensions scale factorially with the number of spin orbitals and electrons.
- Consequently, classical memory is rapidly exhausted even for relatively small molecular systems.
- This scaling bottleneck necessitates the transition to High Performance Computing architectures and advanced quantum algorithms.

Computational Limits in Practice: Oxygen Molecule

- An exact classical diagonalization was attempted to calculate the ground state energy of an O_2 molecule in its triplet state using the 6-31G* basis set.
- This configuration yields 30 spatial orbitals (i.e. 60 spin orbitals) for the 16 electrons. The Hamiltonian matrix dimension D scales combinatorially:

$$D = \binom{60}{16} \approx 1.48 \times 10^{14}$$

- The full dense matrix requires D x D components, resulting in a computational space of approximately 2.19×10^{28} total elements.
- Executing the solver triggered a fatal Out of Memory error on the compute node when the algorithm attempted to allocate an array of 8.18×10^{12} elements.
- Standard systems allocate 8 bytes per double precision float. This array required 65 TB. Dividing this by 1024^4 translates exactly to a 59.5 TiB RAM requirement, halting the process.

Accelerating Exact Diagonalization

- **Distributed Parallelization:** Rather than holding H in isolated RAM, distributed-memory libraries (accessed via Python wrappers like `mpi4py` or `slepc4py`) partition the matrix across a cluster of network nodes. This enables parallel sparse matrix-vector multiplications across hundreds of CPU cores simultaneously.
- **Algorithmic Optimization (The Davidson Method):** General-purpose solvers are often replaced by the Davidson algorithm. Highly parallelizable and specifically tailored for diagonally dominant matrices, which quantum Hamiltonians almost always are.
- **GPU Acceleration:** Shifting the core linear algebra from the CPU to the GPU leverages thousands of compute cores. By using drop-in Python libraries like `CuPy` instead of `NumPy`, the massive floating-point calculations required for $HC = EC$ can be drastically accelerated within a single machine.

Thank You